

syqwq ACM template

Contents

syqwq ACM template	1
1. General	3
2. Graph theory	4
2.1. basic build graph	4
2.2. tarjan SCC	4
2.3. tarjan eDCC	4
2.4. tarjan vDCC	5
2.5. LCA	5
3. Math	6
3.1. Number theory	6
3.1.1. Euler sieve	6
3.1.2. Eratosthenes sieve	6
3.1.3. gcd	6
3.1.4. exgcd	6
3.1.5. lowbit	6
3.1.6. popcount	6
3.1.7. $\varphi(n)$	6
3.1.8. $a \cdot a^{-1} \equiv 1(\text{mod } p)$	7
3.1.9. extended euler theorem	7
3.1.10. linear mod equation system	7
3.1.11. Inclusion-Exclusion principle	7
3.1.12. multiplicative function	8
3.1.13. Möbius inversion	8
3.1.14. Dirichlet convolution	9
3.2. Linear algebra	9
3.2.1. matrix multiplication	9
3.2.2. Gauss elimination	10
3.2.3. linear basis	11
3.3. Polynomial	12
3.3.1. Generating function	12
3.3.2. polynomial brute multiplication	12
3.3.3. FFT	13
3.3.4. Lagrange interpolation	16
3.3.5. primitive root	17
3.3.6. Discrete logarithm / index	19
3.3.7. NTT	19
3.3.8. MTT	20
3.3.9. Newton's Method	22
3.3.9.1. poly inverse	22
3.3.9.2. poly square root	23
3.3.9.3. poly derivative	23
3.3.9.4. poly integral	23
3.3.9.5. poly composition	23
3.3.9.6. poly logarithm	23
3.3.9.7. poly exp	23
3.4. Combinatorics	25
3.4.1. binomial inversion	25
3.4.2. Stirling number of the 2nd kind	26
3.4.3. partition number	27
3.4.4. Stirling number of the 1st kind	27

4. References	28
---------------------	----

1. General

缺省源

```
#include <bits/stdc++.h>
#define pb push_back
#define vc vector
#define fi first
#define se second
#define all(x, n) (x) + 1, (x) + 1 + n
#define NL "\n"
#define NN " "
using namespace std;
typedef long long ll;
typedef vc<ll> vi;
typedef pair<ll, ll> PII;
typedef vc<PII> vpii;
template<class T, class S>
bool chmax(T& x, S y){ return x<y?x=y,1:0; }
template<class T, class S>
bool chmin(T& x, S y){ return x>y?x=y,1:0; }
// #define CF
// #define int long long
// =====

void solve() {
}
// =====
signed main() {
    ios::sync_with_stdio(0);
    cin.tie(0), cout.tie(0);
    cout.setf(ios::fixed), cout.precision(5);
    int T = 1;
#ifdef CF
    cin >> T;
#endif // CF
    while (T--) solve();
    return 0;
}
```

2. Graph theory

2.1. basic build graph

```
vpii e[N];
void add(int a,int b,int c){e[a].pb({b,c});}
// =====
int idx=0,h[N],ne[M],to[M],w[M];
void add(int a,int b,int c){
    w[++idx]=c,to[idx]=b,ne[idx]=h[a],ha[a]=idx;
}
```

2.2. tarjan SCC

有向图中的 scc 要求当前点能够到达所有点，并且其他点能到当前点，于是利用这个思想，在 dfs 遍历的时候按照时间戳压栈，那么在上面的点就是他到的点（如果维护一定性质的话）直到遍历完，如果发现当前点就是这个连通块的最高的点（也就是 dfn 最小的点），那就把这个 scc 中的点全部出栈。scc 缩点之后，得到的是拓扑图。

```
vi e[N];
int dn = 0, dfn[N], low[N], stc[N], top = 0;
int col[N], cn = 0, sz[N];

void scc(int id) {
    low[id] = dfn[id] = ++dn;
    stc[++top] = id, ins[id] = 1;
    for (int it : e[id]) {
        if (!dfn[it]) {
            scc(it), chmin(low[id], low[it]);
        } else if (ins[it]) chmin(low[id], dfn[it]);
    }
    if (low[id] == dfn[id]) {
        cn++;
        int x;
        do {
            col[x = stc[top--]] = cn, ins[x] = 0, sz[cn]++;
        } while (x != id);
    }
}
```

2.3. tarjan eDCC

解决方法是记录边的编号。对于 vector，在 push_back 时将当前边的编号一并压入。对于链式前向星，使用成对变换技巧：初始化 cnt = 1，每条边及其反边在链式前向星中存储的编号分别为 2k 和 2k+1，将当前边编号异或 1 即得反边编号。时间复杂度 $O(n + m)$ 边双缩点之后，得到的是连通分量作为节点的树。

```
int dfn[N], low[N], dn = 0;
int stc[N], top = 0;
int cn = 0, col[M];

void form(int id) {
    cn++;
    for (int x = 0; x != id; x = stc[top--]) col[x] = cn;
}

void dcc(int id, int eid) {
    stc[++top] = id, dfn[id] = low[id] = ++dn;
    for (auto _ : e[id]) {
        if (_.se == eid) continue;
        int it = _.fi;
        if (!dfn[it]) {
            dcc(it, _.se);
            chmin(low[id], low[it]);
            if (low[it] > low[id]) form(it);
        } else chmin(low[id], dfn[it]);
    }
    if (!eid) form(id);
}
```

2.4. tarjan vDCC

还不会 呜呜呜

2.5. LCA

倍增法，先预处理深度和倍增祖先，然后先跳到一起，再一起跳。时间复杂度 $O(n)$ 预处理, $O(\log n)$ 查询

```
int dep[N], anc[N][20];
namespace ZX { // zu xian
void bfs(int root) {
    queue<int> q;
    dep[root] = 1, q.push(root);
    while (q.size()) {
        int id = q.front();
        q.pop();
        for (auto it : e[id]) {
            if (dep[it]) continue;
            dep[it] = dep[id] + 1;
            anc[it][0] = id;
            rep(i, 1, 19) anc[it][i] = anc[anc[it][i - 1]][i - 1];
            q.push(it);
        }
    }
}
int lca(int x, int y) {
    if (dep[x] < dep[y]) swap(x, y);
    per(i, 19, 0) if (dep[anc[x][i]] >= dep[y]) x = anc[x][i];
    if (x == y) return x;
    per(i, 19, 0) if (anc[x][i] != anc[y][i]) x = anc[x][i], y = anc[y][i];
    return anc[x][0];
}
} // namespace ZX
```

3. Math

3.1. Number theory

3.1.1. Euler sieve

线性筛，时间复杂度 $O(n)$

```
vi primes;
bool st[N];
void ss(int maxn){
    for(int i=2;i<=maxn;i++){
        if(!st[i]) primes.pb(i);
        for(int j:primes){
            if(j>maxn/i) break;
            st[i*j]=1;
            if(i%j==0) break;
        }
    }
}
```

3.1.2. Eratosthenes sieve

埃式筛，时间复杂度 $O(n \log \log n)$ ，修改内层循环可以做区间筛

```
vi primes;
bitset<N> st;
void ass(int maxn){
    for(int i=2;i<=maxn;i++){
        if(st[i]) continue;
        primes.pb(i);
        for(int j=i*2;j<=maxn;j+=i) st[j]=1;
    }
}
```

3.1.3. gcd

$(a, b) = (a, b - a) = (b, a - b)$ ，时间复杂度 $O(\log \min\{a, b\})$

```
ll gcd(ll x, ll y){ return y ? x : gcd(y, x%y); }
```

也可以，cpp 内置 `__gcd(x, y)`

3.1.4. exgcd

方程 $ax + by = \gcd(a, b)$ 的一组特解，通解是 $(x, y) = (x_0 + k \cdot \frac{b}{(a, b)}, y_0 - k \cdot \frac{a}{(a, b)}), k \in \mathbb{Z}$

```
void exgcd(int a, int b, int &x, int &y) {
    if(!b) return x = 1, y = 0, void();
    exgcd(b, a % b, y, x), y -= a / b * x;
}
```

3.1.5. lowbit

```
ll lowbit(ll x){ return x&(-x); }
```

3.1.6. popcount

```
ll count(ll x){ return __builtin_popcountll(x); }
int count(int x){ return __builtin_popcount(x); }
```

3.1.7. $\varphi(n)$

欧拉函数 $\varphi(n) = \sum_{i=1}^n [\gcd(i, n) = 1]$.

由容斥原理可推出： $n = p_1^{\alpha_1} \cdot p_2^{\alpha_2} \cdots p_j^{\alpha_j} \Rightarrow \varphi(n) = n \cdot \prod_{i=1}^j (1 - p_i^{-1}) = (p_1 - 1)(p_2 - 1) \cdots (p_j - 1) \cdot p_1^{\alpha_1 - 1} \cdot p_2^{\alpha_2 - 1} \cdots p_j^{\alpha_j - 1}$. 且若 p 为素数，有 $\varphi(p) = p - 1$.

可以在 $O(\log n)$ 求单点欧拉函数，可以在 $O(n)$ 递推求 1 到 n 欧拉函数.

```
// $O(\log n)$ 求单点欧拉函数
ll get_phi(ll x){
    ll phi=1;
```

```

for(ll i=2;i<=x/i;i++){
    if(x%i==0){
        phi*=(i-1);
        x/=i;
        while(x%i==0) phi*=i,x/=i;
    }
}
if(x>1) phi*=(x-1);
return phi;
}
// $O(n)$ 递推求 $1$ 到 $n$ 欧拉函数
bool st[N];
vi primes;
ll phi[N];
void get_phi(ll maxn){
    phi[1]=1;
    for(ll i=2;i<=maxn;i++){
        if(!st[i]) primes.pb(i),phi[i]=i-1;
        for(ll j:primes){
            if(j>maxn/i) break;
            st[i*j]=1;
            if(i%j==0) { phi[i*j]=phi[i]*j; break; }
            phi[i*j]=phi[i]*(j-1);
        }
    }
}
}

```

3.1.8. $a \cdot a^{-1} \equiv 1(\text{mod } p)$

单点求乘法逆元：可以直接解不定方程， $ab \equiv 1(\text{mod } p) \Leftrightarrow ab = mp + 1 \Leftrightarrow ab - mp = 1$ ，有解的充要条件是 $(a, p) = 1$ 。也可以由欧拉定理 $(n, a) = 1 \Rightarrow a^{\varphi(n)} \equiv 1(\text{mod } n)$ ，特别的，若 p 为素数，则 $a^{p-1} \equiv 1(\text{mod } p)$ 。

递推求 1 到 n 的乘法逆元：求 i 的逆元，考虑 $p = \lfloor \frac{p}{i} \rfloor \cdot i + p \% i \equiv 0(\text{mod } p)$ ，其中注意到 $p \% i < i$ ，于是可以递推。 $\lfloor \frac{p}{i} \rfloor \cdot i \equiv -(p \% i) \Leftrightarrow i^{-1} \equiv -(p \% i)^{-1} \cdot \lfloor \frac{p}{i} \rfloor (\text{mod } p)$ 。

3.1.9. extended euler theorem

欧拉定理： $(a, m) = 1 \Rightarrow a^{\varphi(m)} \equiv 1(\text{mod } m)$

扩展欧拉定理： $a^c \equiv a^{c \% \varphi(m) + \varphi(m)}(\text{mod } m)$ if $c \geq \varphi(m)$ ，在这里不要求 $(a, m) = 1$

3.1.10. linear mod equation system

中国剩余定理 和 两两相消。考虑方程组

$$\begin{cases} x \equiv a_1(\text{mod } m_1) \\ x \equiv a_2(\text{mod } m_2) \\ \dots \\ x \equiv a_k(\text{mod } m_k) \end{cases}$$

中国剩余定理：如果 $m_1, m_2, \dots, m_k$ 两两互素，则有 $x \equiv \sum_{i=1}^k M_i' M_i a_i (\text{mod } M)$ ，其中， $M = \prod_{i=1}^k m_i$ ， $M_i = M/m_i$ ， $M_i' M_i \equiv 1(\text{mod } m_i)$

3.1.11. Inclusion-Exclusion principle

最简单的形式，便于理解：

$$|S - A \cup B| = |S| - |A| - |B| + |A \cap B|$$

推广：我们将满足某种性质记作 a_i ，不满足某种性质记作 $1 - a_i$ ，则有

$$N((1 - a_1)(1 - a_2) \dots (1 - a_n)) = \sum_{k=0}^n (-1)^k \sum_{1 \leq j_1 \leq \dots \leq j_k \leq n} N(a_{j_1} \dots a_{j_k})$$

如果选 k 种性质进行容斥是相互等价的，还可以写成：

$$N((1-a_1)(1-a_2)\cdots(1-a_n)) = \sum_{k=0}^n (-1)^k \binom{n}{k} N(a_1 a_2 \cdots a_k)$$

可以考虑 dp，如果要枚举集合可以使用 dfs，注意记录 -1 的符号

3.1.12. multiplicative function

积性函数：若对于 $f(x)$ ，有 $p \perp q \Rightarrow f(pq) = f(p)f(q)$ ，则称 f 为积性函数

如果不要求 $p \perp q$ ，则称为完全积性函数

对于积性函数，可以利用欧拉筛， $O(n)$ 递推他们的值

```
// 完全积性函数
bool st[N];
vi primes;
ll f[N];
void ss(int maxn){
    for(int i=2;i<=maxn;i++){
        if(!st[i]) primes.pb(i),f[i]=initial_value(i);
        for(int j:primes){
            if(j>maxn/i) break;
            st[i*j]=1,f[i*j]=f[i]*f[j];
            if(i%j==0) break;
        }
    }
}
// 求 p perp q 的积性函数
// 单点求
ll get_f(ll n){
    ll ans=1;
    for(int i=2;i<=n/i;i++){
        int cnt=0;
        while(n%i==0) cnt++,n/=i;
        ans=ans*f(i,cnt)%mod; // f(p,k)=f(p^k)
    }
    if(n>1) ans=ans*f(n,1)%mod;
    return ans;
}
// 也可以利用最小的质数来递推 1 ~ n
// cnt[] 记录最小质数出现的次数
bool st[N];
vi primes;
ll f[N],cnt[N];
void ss(int maxn){
    f[1]=1;
    for(int i=2;i<=maxn;i++){
        if(!st[i]) primes.pb(i),cnt[i]=1,f[i]=calc_f(i,1);
        for(int j:primes){
            if(j>maxn/i) break;
            st[i*j]=1;
            if(i%j==0){
                cnt[i*j]=cnt[i]+1;
                f[i*j]=f[i]/calc_f(j,cnt[i])*calc_f(j,cnt[i]+1);
                break;
            }
            cnt[i*j]=1;
            f[i*j]=f[i]*calc_f(j,1);
        }
    }
}
```

3.1.13. Möbius inversion

对于数论函数，常见的两种莫比乌斯反演的两种形式：

- 对因子反演： $f(n) = \sum_{d|n} g(d) \Leftrightarrow g(n) = \sum_{d|n} \mu\left(\frac{n}{d}\right) f(d) = \sum_{dm|n} \mu(d) f\left(\frac{n}{d}\right)$ (后面的等号是由于因数的成对出现)

- 对倍数反演: $f(d) = \sum_{\{d|n, n \leq N\}} g(n) \Leftrightarrow g(d) = \sum_{\{d|n, n \leq N\}} \mu\left(\frac{n}{d}\right) f(n)$

本质是在整除的意义上划分出的集合上进行容斥。其中 μ 为莫比乌斯函数:

$$\mu(x) = \begin{cases} 1 & x = 1 \\ (-1)^k & x = p_1 p_2 \cdots p_k \\ 0 & x = p_1^{\alpha_1} p_2^{\alpha_2} \cdots p_k^{\alpha_k} \end{cases}$$

μ 为积性函数, 利用欧拉筛可以 $O(n)$ 递推求

```
bool st[N];
vi primes;
int mu[N];
void get_mu(int maxn){
    mu[1]=1;
    for(int i=2;i<=maxn;i++){
        if(!st[i]) primes.pb(i),mu[i]=-1;
        for(int j:primes){
            if(j>maxn/i) break;
            st[i*j]=1;
            if(i%j==0){ mu[i*j]=0; break; }
            mu[i*j]=-mu[i];
        }
    }
}
```

一般单项不好求, 但是如果对于倍数的式子相加, 或者因数的式子相加的和 (就是考虑一个集合的时候) 好求, 可以考虑整体求, 然后对求和的函数进行反演。

3.1.14. Dirichlet convolution

设 $f: \mathbb{N} \rightarrow \mathbb{R}, g: \mathbb{N} \rightarrow \mathbb{R}$, 则定义他们的狄利克雷卷积为 $(f * g)(n) = \sum_{d|n} f(d)g\left(\frac{n}{d}\right)$.

若 f 和 g 为积性函数, 则他们的卷积也为积性函数, 且满足交换律, 结合律。

为了方便, 我们定义如下函数:

- $1(n) = 1$, 在狄利克雷卷积的乘法中与 μ 互为逆元
- $\varepsilon(n) = [n = 1]$, 狄利克雷卷积的乘法单位元
- $\text{id}(n) = n$

于是, 我们有:

- $f = g * 1 \Leftrightarrow g = f * \mu$ (aka. Möbius inversion)
- $\varepsilon = \mu * 1 \Leftrightarrow \mu = \varepsilon * \mu$, 证明可以考虑右侧反演单位元或者左侧的话其实就是 $(1 + (-1))^k$ 的二项式展开
- $\text{id} = \varphi * 1 \Leftrightarrow \varphi = \text{id} * \mu$

一些技巧:

- $\sum_{i=1}^n i \cdot [i \perp n] = \frac{1}{2}(n\varphi(n) + \varepsilon(n))$

Proof 考虑到 $\gcd(i, n) = \gcd(n - i, n)$, 所以他们是成对出现的, 一共的对数就是 $\varphi(n)$, 每一对贡献的和是 n , 特判 $n = 1$ 的情况。

3.2. Linear algebra

3.2.1. matrix multiplication

可以用来加速 线性 dp 的递推。数据结构中, 将维护值扩展成维护矩阵。

$A \in M_{n \times t}(\mathbb{R}), B \in M_{t \times m}(\mathbb{R})$, 那么 $AB[i, j] = \sum_{k=1}^t A[i, k] \cdot B[k, j]$.

```
const int N=105;
const ll mod=1e9+7;
template<class T>
struct mat{
    int n,m; // 0~n-1, 0~m-1
    T a[N][N];
    mat(int n,int m) { this->n=n,this->m=m; memset(a, 0, sizeof a); }
```

```

T* operator[](int x) { return a[x]; }
const T* operator[](int x) const { return a[x]; }
mat operator-(const mat &rhs) const {
    mat<T> res(n,m);
    for(int i=0;i<n;i++) for(int j=0;j<m;j++)
        res[i][j]=(a[i][j]+mod-rhs[i][j])%mod;
    return res;
}
mat operator+(const mat &rhs) const {
    mat<T> res(n,m);
    for(int i=0;i<n;i++) for(int j=0;j<m;j++)
        res[i][j]=(a[i][j]+rhs[i][j])%mod;
    return res;
}
mat operator*(const mat &rhs) const {
    // assert(m==rhs.n);
    mat<T> res(n,rhs.m);
    for(int i=0;i<n;i++) for(int j=0;j<rhs.m;j++) for(int k=0;k<m;k++)
        res[i][j]+=a[i][k]*rhs[k][j], res[i][j]%=mod;
    return res;
}
mat qmi(int k) {
    // assert(n==m);
    mat<T> res(n, n), t=*this;
    for(int i=0;i<n;i++) res[i][i]=1;
    for(;k;t=t*t,k>>=1) if(k&1) res=res*t;
    return res;
}
};

```

3.2.2. Gauss elimination

线性方程组 $Ax = b$ 的解。设 A 的增广矩阵记作 $\tilde{A} = [A \mid b]$ ，将解集记作 $S = \{x \mid Ax = b\}$ ，则有

- $|S| = 0 \Leftrightarrow \text{rank} A < \text{rank } \tilde{A}$
- $|S| = 1 \Leftrightarrow \text{rank} A = \text{rank } \tilde{A}$
- $|S| = \aleph_1 \Leftrightarrow \text{rank} A > \text{rank } \tilde{A}$

高斯消元时间复杂度 $O(n^{\{2\}}m)$

```

constexpr int N=105;
constexpr double eps=1e-6;
template<class T=double>
struct gmat{
    int n,m; // 0~n-1, 0~m-1
    T a[N][N];
    gmat(int n,int m) { this->n=n,this->m=m; memset(a, 0, sizeof a); }
    T* operator[](int x) { return a[x]; }
    const T* operator[](int x) const { return a[x]; }
    // 0: no solution || 1: one solution || -1: inf solution
    int gauss(){
        int c,r;
        for(c=0,r=0;c<n;c++){
            int t=r;
            for(int i=r+1;i<n;i++) if(fabs(a[t][c])<fabs(a[i][c])) t=i;
            if(fabs(a[t][c])<eps) continue;
            for(int i=c;i<m;i++) swap(a[t][i],a[r][i]);
            for(int i=m-1;i>=c;i--) a[r][i]/=a[r][c];
            for(int i=r+1;i<n;i++)
                if(fabs(a[i][c])>eps)
                    for(int j=m-1;j>=c;j--) a[i][j]-=a[i][c]*a[r][j];
            r++;
        }
        if(r<n){
            for(int i=r;i<n;i++) if(fabs(a[i][m-1])>eps) return 0;
            return -1;
        }
        for(int i=n-1;i>=0;i--) for(int j=0;j<i;j++) a[j][m-1]-=a[i][m-1]*a[j][i];
        return 1;
    }
};

```

```

    }
};

```

求解异或方程组在时间复杂度要求不严格的情况下，可以使用 bit 结构体：

```

// 记得修改 gauss 中关于精度的判断
struct bit {
    bool val;
    bit(bool val=0):val(val) {}
    bit operator+(const bit& x) const { return bit(val^x.val); }
    bit operator-(const bit& x) const { return bit(val^x.val); }
    void operator-=(const bit& x) { val^=x.val; }
    bit operator*(const bit& x) const { return bit(val&x.val); }
    bit operator/(const bit& x) const { return bit(val); }
    void operator/=(const bit& x) { val=val/x.val; }
    bool operator<(const bit& x) const { return !val&&x.val; }
    bool operator==(const int x) const { return val==x; }
    bool operator!=(const int x) const { return !(*this==x); }
    operator bool() const { return val; }
    friend istream& operator>>(istream& in, bit& b){ return in>>b.val; }
    friend ostream& operator<<(ostream& out, const bit& b){ return out<<b.val; }
};

```

或者，使用 bitset 优化，但注意读入需要先读进一个 bool 变量，然后再赋值。时间复杂度 $O\left(\frac{n^2 m}{\omega}\right)$

```

constexpr int N = 105;
struct bmat {
    int n, m;
    bitset<N> a[N];
    bmat(int n, int m) {
        this->n = n, this->m = m;
        for (int i = 0; i < n; i++) a[i] &= 0;
    }
    bitset<N> &operator[](int x) { return a[x]; }
    const bitset<N> &operator[](int x) const { return a[x]; }
    int gauss() {
        int c, r;
        for (r = c = 0; c < n; c++) {
            int t = r;
            for (; t < n && !a[t][c]; t++);
            if (!a[t][c]) continue;
            swap(a[t], a[r]);
            for (int i = r + 1; i < n; i++) if (a[i][c]) a[i] ^= a[r];
            r++;
        }
        if (r < n) {
            for (int i = r; i < n; i++) if (a[i][m - 1]) return 0;
            return -1;
        }
        for (int i = n - 1; i >= 0; i--)
            for (int j = 0; j < i; j++)
                a[j][m - 1] = a[j][m - 1] ^ a[i][m - 1] & a[j][i];
        return 1;
    }
};

```

3.2.3. linear basis

布尔域 $\mathbb{Z}_2$ 和 异或（加法），逻辑与（数乘）构成线性空间，其中的极大线性无关组称为**线性基**。 $p[i]$ 表示最高位是 i 位的基向量。

```

typedef unsigned long long ull;
constexpr int N=105;
constexpr int B=50;
int rk=0;
ull p[N];
void insert(ull x){
    for(int i=B;i>=0;i--){
        if(!(x>>i&1)) continue;

```

```

        if(!p[i]) return p[i]=x,++rk,void();
        x^=p[i];
    }
}

```

3.3. Polynomial

3.3.1. Generating function

形式幂级数 $A(x) = \sum_{i \geq 0} a_i x^i$, 记 x^n 的系数为 $[x^n]A(x)$

形式幂级数 $A(x)$ 的逆元: $A(x)B(x) = 1$ 存在的条件是 $[x^0]A(x) \neq 0$

- $A(x) = \frac{1}{1-ax} = \sum_{i \geq 0} a^i x^i$
- $A(x) = \frac{1}{(1-x)^k} = \sum_{i \geq 0} \binom{i+k-1}{i} x^i$

对于**组合型枚举**, 设 $S = \{a_1, a_2, \dots, a_k\}$, 且 a_i 可以取的次数的集合为 M_i , 记 $F_i(x) = \sum_{u \in M_i} x^u$, 则从 S 中取 n 个元素组成的集合的方案数 $g(n)$ 的常生成函数 $G(x) = \sum_{i \geq 0} g(i) x^i$, 满足

$$G(x) = F_1(x)F_2(x) \cdots F_k(x)$$

对于 EGF, 设 $f_1(i)$ 表示第一种物品选 i 个的排列方案, $f_2(i)$ 表示第二种物品选 i 个的排列方案, $g(i)$ 表示使用前面两种物品一共 i 个的排列方案, 则

$$g(n) = \sum_{i=0}^n \binom{n}{i} f_1(i) f_2(n-i) \Leftrightarrow \frac{g(n)}{n!} = \sum_{i=0}^n \frac{f_1(i)}{i!} \frac{f_2(n-i)}{(n-i)!}$$

除以 $n!$ 的作用就是暂时“抵消”掉这 n 个对象自身的排列属性, 使得它们在数学上更容易组合。当我们把两个 EGF 相乘时, 这个被抵消的排列属性会以一种极其优美的方式被重新组合起来。 $g_n = \sum_{i=0}^n \binom{n}{i} f_1(i) f_2(n-i)$ 代表: 将 n 个有标号的对象分成两部分, 第一部分构成 f_1 结构, 第二部分构成 f_2 结构的方案总数。

对于**排列型枚举**, 设 $S = \{a_1, a_2, \dots, a_k\}$, 且 a_i 可以取的次数的集合为 M_i , 记 $F_i(x) = \sum_{u \in M_i} \frac{x^u}{u!}$, 则从 S 中取 n 个元素排成一列的方案数 $g(n)$ 的指数生成函数 $G(x) = \sum_{i \geq 0} g(i) \frac{x^i}{i!}$, 满足

$$G(x) = F_1(x)F_2(x) \cdots F_k(x)$$

- $\exp(ax) = 1 + ax + a^2 \frac{x^2}{2!} + \dots = \sum_{n \geq 0} a^n \frac{x^n}{n!}$
- $\frac{1}{2}(\exp(x) + \exp(-x)) = 1 + \frac{x^2}{2!} + \frac{x^4}{4!} + \dots$
- $\ln(1+x) = \sum_{n \geq 1} \frac{(-1)^{n+1}}{n} x^n$
- $-\ln(1-x) = \sum_{n \geq 1} \frac{1}{n} x^n$

3.3.2. polynomial brute multiplication

形式幂级数 $A(x) = \sum_{i \geq 0} a_i x^i, B(x) = \sum_{i \geq 0} b_i x^i$, 则定义他们的乘积为 $AB(x) = \sum_{i \geq 0} \left(\sum_{s+t=i} a_s b_t \right) x^i = \sum_{i \geq 0} \left(\sum_{j=0}^i a_j b_{i-j} \right) x^i$. 暴力计算:

```

struct poly {
    vpii a;
    void init(bool id = 0) {
        a.clear();
        if (id) a.emplace_back(0, 1);
    }
    poly operator*(const poly &p) const {
        poly ret; ret.init();
        map<int, int> mp;
        mp.clear();
        for (auto i : a)
            for (auto j : p.a)
                mp[i.fi + j.fi] += i.se * j.se;
        for (auto i : mp) ret.a.push_back(i);
        sort(ret.a.begin(), ret.a.end());
        return ret;
    }
};

```

3.3.3. FFT

多项式在点值表示下，乘法的时间复杂度是 $O(n)$ ，因此我们考虑将多项式先变成点值表示，这个过程叫做 DFT，他的核心思想是对于这 n 个点的取值的选取。

对于多项式 $A(x)$ ，我们将他的偶数项之和记作 $A_0(x)$ ，将奇数项之和记作 $A_1(x)$ ，注意此处的两个部分和的次数是 $\deg A_0(x) = \deg A_1(x) = \frac{1}{2} \deg A(x) = \frac{n}{2}$ ，那么，我们有

$$\begin{aligned} A(x) &= A_0(x^2) + xA_1(x^2) \\ A(-x) &= A_0(x^2) - xA_1(x^2) \end{aligned}$$

注意到这是一个分治的过程。因为每次次数折半，且为了保证每次平方复数的模长不会指数爆炸，我们考虑使用 n 次单位根 $\omega_n^k = e^{i\frac{2\pi}{n}k}$ ，他有如下性质：

- $\omega_n^{2k} = \omega_{n/2}^k$
- $\omega_n^{k+n/2} = -\omega_n^k$

于是，在 $k < \frac{n}{2}$ 时，DFT 变成：

$$\begin{aligned} A(\omega_n^k) &= A_0(\omega_{n/2}^k) + \omega_n^k A_1(\omega_{n/2}^k) \\ A(\omega_n^{k+n/2}) &= A_0(\omega_{n/2}^k) - \omega_n^k A_1(\omega_{n/2}^k) \end{aligned}$$

在每一次枚举 $\omega_{n/2}^k$ 的值的时候，我们可以一下得到两组点的值。且次数每次折半，而求值最多需要 n 个值，所以时间复杂度为 $O(n \log n)$ 。

假设 $A(x) = \sum_{i=0}^{n-1} a_i x^i$ ，那么对于 DFT，他的矩阵表示为：

$$\mathcal{F} = \begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega_n^1 & \omega_n^2 & \dots & \omega_n^{n-1} \\ 1 & \omega_n^2 & \omega_n^4 & \dots & \omega_n^{2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_n^{n-1} & \omega_n^{2(n-1)} & \dots & \omega_n^{(n-1)(n-1)} \end{bmatrix}$$

于是我们有：

$$\begin{bmatrix} p_0 \\ p_1 \\ p_2 \\ \vdots \\ p_{n-1} \end{bmatrix} = \mathcal{F} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_{n-1} \end{bmatrix}$$

那么，接下来我们得到了点值表示，我们还需要把它最后变成系数表示，这个过程叫做 IDFT，也就是根据 $[p_i]$ 求系数 $[a_i]$ 。由于这个变换 $\mathcal{F}$ 的矩阵是一个 Vandemort 矩阵，可逆当且仅当 ω_n^k 互不相同，这是显然的。考虑算出这个变换矩阵的逆矩阵，于是

$$\begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_{n-1} \end{bmatrix} = \mathcal{F}^{-1} \begin{bmatrix} p_0 \\ p_1 \\ p_2 \\ \vdots \\ p_{n-1} \end{bmatrix}$$

其中，

$$\mathcal{F}^{-1} = \frac{1}{n} \begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega_n^{-1} & \omega_n^{-2} & \dots & \omega_n^{-(n-1)} \\ 1 & \omega_n^{-2} & \omega_n^{-4} & \dots & \omega_n^{-2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_n^{-(n-1)} & \omega_n^{-2(n-1)} & \dots & \omega_n^{-(n-1)(n-1)} \end{bmatrix}$$

因此发现, IDFT 只是将 DFT 中的 ω_n^k 变成 ω_n^{-k} , 同时前面乘上了 $\frac{1}{n}$, 时间复杂度也是 $O(n \log n)$

注意这是一个一生二, 二生四 的不断开平方根的过程

复数运算

```
struct Complex {
    double r, i;
    Complex(double r = 0, double i = 0) : r(r), i(i) {}
    Complex operator+(const Complex &p) const { return Complex(r + p.r, i + p.i); }
    Complex operator-(const Complex &p) const { return Complex(r - p.r, i - p.i); }
    Complex operator*(const Complex &p) const { return Complex(r * p.r - i * p.i, r * p.i + i * p.r); }
    void operator+=(const Complex &p) { r += p.r, i += p.i; }
    void operator*=(const Complex &p) {
        double t = r;
        r = r * p.r - i * p.i, i = t * p.i + i * p.r;
    }
};
```

递归实现

```
constexpr int N = 4e6 + 5;
const double PI = acos(-1);
int n, m;
Complex f[N], g[N];
void FFT(Complex a[], int lim, int sign) { // lim=2^k
    if (lim == 1) return;
    Complex a0[lim >> 1], a1[lim >> 1];
    for (int i = 0; i < lim; i += 2) a0[i >> 1] = a[i], a1[i >> 1] = a[i + 1];
    FFT(a0, lim >> 1, sign), FFT(a1, lim >> 1, sign);
    Complex wn(cos(2 * PI / lim), sign * sin(2 * PI / lim)), w(1, 0);
    for (int i = 0; i < (lim >> 1); i++, w *= wn) {
        Complex t = w * a1[i];
        a[i] = a0[i] + t, a[i + (lim >> 1)] = a0[i] - t;
    }
}
void solve() {
    int limit = 1; while (limit <= n + m) limit <<= 1;

    FFT(f, limit, 1), FFT(g, limit, 1);
    for (int i = 0; i < limit; i++) f[i] *= g[i];
    FFT(f, limit, -1);

    for (int i = 0; i <= n + m; i++) cout << (int)(f[i].r / limit + 0.5) << " ";
}
```

蝴蝶操作优化

观察向下递归的序列

```
0-> 0 1 2 3 4 5 6 7
1-> 0 2 4 6|1 3 5 7
2-> 0 4|2 6|1 5|3 7
end 0|4|2|6|1|5|3|7
```

最底层是数字的二进制刚好是开始系数的逆序, 于是可以预处理每个数字在长度为 L 下的二进制的逆序, 然后交换, 再递推即可

递推实现

```
constexpr int N = 4e6 + 5;
const double PI = acos(-1);
int n, m;
Complex f[N], g[N];
int limit = 1, L = 0, rev[N]; // limit=min(2^k)>n+m=deg(f'*g')+1, L=log_2(limit)
void FFT(Complex a[], int lim, int sign) { // lim=2^k
    for (int i = 0; i < lim; i++)
        if (i < rev[i]) swap(a[i], a[rev[i]]);
    Complex wn, w, x, y;
    // mid 为区间长度的一半, 因为刚好可以和指数上的 2 pi 约掉
```

```

// 同时方便后续蝴蝶操作的 offset 是 区间长度的一半
for (int mid = 1; mid < lim; mid <= 1) {
    wn = Complex(cos(PI / mid), sign * sin(PI / mid));
    for (int r = mid < 1, j = 0; j < lim; j += r) {
        w = Complex(1, 0);
        for (int k = 0; k < mid; k++, w *= wn) {
            x = a[j + k], y = w * a[j + mid + k];
            a[j + k] = x + y, a[j + mid + k] = x - y;
        }
    }
}
if (sign == -1) {
    for (int i = 0; i < lim; i++) a[i].r /= lim, a[i].i /= lim;
}
}

void convolve(Complex f[], Complex g[], int df, int dg) {    // f = f * g
    limit = 1, L = 0;
    while (limit <= df + dg) limit <= 1, L++;
    for (int i = df + 1; i < limit; i++) f[i] = Complex(0, 0);
    for (int i = dg + 1; i < limit; i++) g[i] = Complex(0, 0);
    // rev[11001]=1<<(L-1)+'rev[1100]'=1<<(L-1)+rev[01100] 注意有前导0, 所以要把他右移去掉
    for (int i = 0; i < limit; i++) rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (L - 1));
    FFT(f, limit, 1), FFT(g, limit, 1);
    for (int i = 0; i < limit; i++) f[i] *= g[i];
    FFT(f, limit, -1);
}

void solve() {
    convolve(f, g, n, m);
    for (int i = 0; i <= n + m; i++) cout << (int)(f[i].r + 0.5) << " ";
}

```

使用 FFT 的更完善的多项式模板

```

////////// FTT poly //////////
namespace Poly {
using cd = complex<double>;
const double PI = acos(-1.0);
constexpr int N = 4e6 + 5;

void FFT(vc<cd> &a, int lim, int sign, const ll rev[]) {
    for (int i = 0; i < lim; i++)
        if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int mid = 1; mid < lim; mid <= 1) {
        cd wn(cos(PI / mid), sign * sin(PI / mid));
        for (int r = mid < 1, j = 0; j < lim; j += r) {
            cd w(1, 0);
            for (int k = 0; k < mid; k++, w *= wn) {
                cd x = a[j + k];
                cd y = w * a[j + mid + k];
                a[j + k] = x + y;
                a[j + mid + k] = x - y;
            }
        }
    }
    if (sign == -1) {
        for (int i = 0; i < lim; i++)
            a[i] /= lim;
    }
}

using poly = vc<ll>;
poly get(int deg) { return poly(deg + 1, 0); }
poly operator+(poly f, poly g) {
    int d = max(f.size(), g.size());
    f.resize(d), g.resize(d);
    poly res = get(d - 1);
    for (int i = 0; i < d; ++i) res[i] = f[i] + g[i];
    return res;
}
}

```

```

poly operator-(poly f, poly g) {
    int d = max(f.size(), g.size());
    f.resize(d), g.resize(d);
    poly res = get(d - 1);
    for (int i = 0; i < d; ++i) res[i] = f[i] - g[i];
    return res;
}
poly operator*(poly f, poly g) {
    static ll rev[N];
    int df = f.size() - 1, dg = g.size() - 1, limit = 1, L = 0;
    while (limit <= df + dg) limit <= 1, L++;
    for (int i = 0; i < limit; i++) rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (L - 1));
    vc<cd> fa(f.begin(), f.end()), fb(g.begin(), g.end());
    fa.resize(limit), fb.resize(limit);
    FFT(fa, limit, 1, rev), FFT(fb, limit, 1, rev);
    for (int i = 0; i < limit; i++) fa[i] *= fb[i];
    FFT(fa, limit, -1, rev);
    poly res = get(df + dg);
    for (int i = 0; i <= df + dg; i++) res[i] = static_cast<ll>(round(fa[i].real()));
    return res;
}
poly operator*(poly f, ll x) {
    for (int i = 0; i < f.size(); i++) f[i] *= x;
    return f;
}
poly mod(poly f, int n) {
    f.resize(n, 0);
    return f;
}
poly inv(poly h) {
    int n = h.size();
    if (n == 0) return poly();
    poly f = get(0), d = get(0);
    f[0] = 1, d[0] = 2;
    for (int len = 2; (len >> 1) < n; len <= 1)
        f = mod(f * (d - mod(h, len) * f), len);
    return f;
}
poly sqrt(poly h) {
    int n = h.size();
    poly f = get(0);
    f[0] = 1;
    for (int len = 2; (len >> 1) < n; len <= 1) {
        poly invf = inv(mod(f, len));
        poly t = mod(h, len) * invf;
        f.resize(len);
        f = (f + t) * 0.5; // New f is (f_old + h/f_old)/2
        f.resize(len);
    }
    return f;
}
} // namespace Poly
using namespace Poly;

```

3.3.4. Lagrange interpolation

已知 $n + 1$ 个点 $(x_i, y_i) \forall i \in \{0, \dots, n\}$, 可以通过拉格朗日插值求出一个 n 阶多项式 $f(x)$, 满足 $f(x_i) = y_i$.

考虑构造函数 $f(x) = \sum_{i=0}^n f_i(x)$, 其中 $f_i(x_j) = [i = j]$, 因此对于 $f_i(x)$, 每个 $x_j (j \neq i)$ 都是他的零点, 所以 $f_i(x) = c_i \cdot \prod_{j \neq i} (x - x_j)$, 代入 $x = x_i$, 可以知道这个待定的系数是 $c_i = y_i \cdot \prod_{j \neq i} (x - x_j)^{-1}$, 因此我们达到了拉格朗日多项式:

$$f(x) = \sum_{i=0}^n f_i(x) = \sum_{i=0}^n y_i \cdot \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$$

拉格朗日插值可以看成是在多项式环上的中国剩余定理 CRT。(感觉有点类似线性对偶基)

- 暴力实现：先计算 $g(x) = \prod_{i=1}^n (x - x_i)$ ，再求 n 次 $g(x)/(x - x_i)$ ，按照权值和将他们相加，时间复杂度是 $O(n^2)$
- 求单点 $x = k$ 的值 $f(k)$ 暴力实现，时间复杂度是 $O(n^2)$

```
constexpr int mod = 998244353;
constexpr int N = 2010;
int n, x[N], y[N];
ll inv(int x) { return qmi(x, mod-2); }
ll lagrange(int k) {
    ll res = 0;
    for (int i = 1; i <= n; i++) {
        ll p = 1, q = 1;
        for (int j = 1; j <= n; j++) {
            if (i == j) continue;
            q = q * (k + mod - x[j]) % mod;
            p = p * (x[i] + mod - x[j]) % mod;
        }
        res = (res + y[i] * q % mod * inv(p) % mod) % mod; // q / p
    }
    return res;
}
```

- $x_i = i$ ，且要求 $f(k)$ ，时间复杂度可以做到 $O(n)$

此时，原式变为 $f(k) = \sum_{i=1}^n y_i \prod_{j \neq i} \frac{k-j}{i-j}$ ，对于分子来说，有 $\prod_{j \neq i} k-j = \frac{1}{k-i} \prod_{j=0}^n k-j$ 可以维护关于 k 的前缀积和后缀积，对于分母来说，可以维护阶乘，这样可以 $O(1)$ 计算 $\prod_{j \neq i} \frac{k-j}{i-j}$ ，从而整体时间复杂度为 $O(n)$

(以下代码为拉插求 $f(n) = \sum_{i=1}^n i^k$ ，可以知道 $\deg f = k+1$ ，所以需要 $k+2$ 个点)

```
const ll mod = 1e9 + 7;
const ll K = 1e6 + 5;
ll n, k; // f(n) deg(f)=k+1
ll y[K], fac[K], pre[K], suf[K];
void solve() {
    fac[0] = pre[0] = suf[k+3] = 1;
    for (int i = 1; i <= k+2; i++) fac[i] = fac[i-1] * i % mod;
    for (int i = 1; i <= k+2; i++) pre[i] = pre[i-1] * (n-i) % mod;
    for (int i = k+2; i > 0; i--) suf[i] = suf[i+1] * (n-i) % mod;
    for (int i = 1; i <= k+2; i++) y[i] = (y[i-1] + qmi(i, k)) % mod;
    ll ans = 0;
    for (int i = 1; i <= k+2; i++) {
        ll q = pre[i-1] * suf[i+1] % mod;
        ll p = (k-i) & 1 ? -1 : 1;
        p = (p + mod) * fac[i-1] % mod * fac[k+2-i] % mod;
        ans = (ans + y[i] * q % mod * inv(p) % mod) % mod;
    }
    cout << (ans + mod) % mod;
}
```

3.3.5. primitive root

在 $\mathbb{Z}_m$ 上，若 $a \perp m$ ，定义整数 a 的阶为 $\min_{\delta \in \mathbb{Z}_+} a^\delta \equiv 1 \pmod{m}$ 。如果 $\delta_m(a) = \varphi(m)$ ，则称 a 为 m 的原根。(原根是可以仅仅通过自己生成整个有限域的东西)

若 $a \perp m, \delta = \delta_m(a)$ ，阶有如下性质：

1. $a^0, a^1, \dots, a^{\delta-1}$ 两两不同。(反证法)
2. $a^\gamma \equiv a^{\gamma'} \pmod{m} \Leftrightarrow \gamma \equiv \gamma' \pmod{\delta}$ (阶是最小的循环节)
3. $\delta \mid \varphi(m)$ 。

考虑反证法，设 $\varphi(m) = q \cdot \delta + r, 0 < r < \delta$ ，则有 $a^{\varphi(m)} \equiv a^{\delta \cdot q + r} \equiv 1 \pmod{m} \Leftrightarrow a^r \equiv 1 \pmod{m}$ ，与原根定义矛盾

4. 若 $\delta_m(a) = g$ ，则 $\delta_m(a^k) = g / \gcd(g, k)$

设 $\delta_m(a^k) = t$ ，则有 $a^{kt} \equiv a^g \equiv 1 \pmod{m} \Leftrightarrow kt \equiv g \equiv 0 \pmod{g}$ ，设 $k = \gcd(k, g) \cdot p_1, g = \gcd(k, g) \cdot$

$p_2, p_1 \perp p_2$, 化简得 $g \mid kt \Leftrightarrow p_2 \mid p_1 \cdot t \Leftrightarrow p_2 \mid t \Leftrightarrow t = q \cdot (g / \gcd(k, g))$ 因为阶要求最小, 所以 $\delta_m a^k = t = g / \gcd(k, g)$.

只有 $2, 4, p^a, 2p^a$ 有原根, 其中 p 为奇素数.

设 $m > 1, g \perp m$, 则 g 为 m 的原根当且仅当 对于任意 $\varphi(m)$ 的质因子 $q_i, g^{\varphi(m)/q_i} \not\equiv 1 \pmod{m}$. 因为 $\delta \mid \varphi(m)$, 若 g 不是原根, 那么 δ 一定是 $\varphi(m)$ 的真因子, 而 $\varphi(m)/q_i$ 涵盖了所有 $\varphi(m)$ 的真因子的倍数. 这样寻找原根最小原根的时间复杂度大约是 $O(\sqrt[4]{n})$.

由于原根可以生成整个有限域的元素, 其余的原根一定也是最小原根 g 的幂次, 由第四条性质若 g 是 m 的原根, 则 g^k 也是原根的充要条件是 $k \perp \varphi(m)$, 因此可以利用这个性质来找出所有原根. 所以, 所有的原根的数量就是 $\sum_{k=1}^{\varphi(m)} [k \perp \varphi(m)] = \varphi(\varphi(m))$. 因此, n 的原根的数量是 $\varphi(\varphi(n))$, 因此整个算法的时间复杂度是 $O(\sqrt[4]{n} \log n)$.

```

///// require /////
// get_phi(), qmi()
////////////////////////////////////
const int N = 1e6 + 5;
ll n, d;
vi primes;
ll phi[N];
#define gcd __gcd
bool exist(ll x) { // 判断 x 是否有原根
    if (x == 2 || x == 4) return 1;
    if (x % 2 == 0) x /= 2;
    for (ll p = 0, i = 1; p <= x; i++) {
        p = primes[i];
        if (x % p == 0) {
            while (x % p == 0) x /= p;
            return x == 1;
        }
    }
    return 0;
}
vi divide(ll x) { // 分解质因数到 fac 中
    vi fac; fac.clear();
    for (ll i = 2; i <= x / i; i++) {
        if (x % i == 0) {
            fac.push_back(i);
            while (x % i == 0) x /= i;
        }
    }
    if (x > 1) fac.push_back(x);
    return fac;
}
ll min_pr(ll x) { // 找到 x 的最小的原根
    auto fac = divide(phi[x]);
    for (ll i = 1; i++;) {
        if (gcd(x, i) != 1) continue;
        bool flg = 1;
        for (ll p : fac)
            if (qmi(i, phi[x] / p, n) == 1) {
                flg = 0;
                break;
            }
        if (flg) return i;
    }
    return 0;
}
vi all_pr(ll g, ll x) { // 根据最小的原根 g 找到所有 x 的原根
    vi pr; pr.clear();
    // gk = g ^ k
    for (ll gk = g, k = 1; pr.size() < phi[phi[x]]; k++, gk = gk * g % x) {
        if (gcd(k, phi[x]) == 1) pr.push_back(gk);
    }
    return pr;
}

```

```

}
void solve() {
    get_phi(1e6);
    if (!exist(n)) return;
    ll g = min_pr(n);
    auto pr = all_pr(g, n);
    cout << phi[phi[n]] << NL; // 原根的数量
    for(ll x:pr) cout << x << NN;
    cout<<NL;
}

```

3.3.6. Discrete logarithm / index

对于质数 p , 假设 g 是 p 的一个原根, 则 $g^0, g^1, \dots, g^{p-2}$ 在模 p 的意义下是 $1, 2, \dots, p-1$ 的一个排列. (一因为他们两两不同, 且没有零元). 假设对于 $1 \leq x < \varphi(p) = p-1$ 有 $g^c \equiv x \pmod{p}$, 则称 x 的**指标/离散对数**为 c , 记作 $\text{ind } x = \text{ind}_g x$.

对于离散对数, 有类似对数的性质:

1. $\text{ind}(xy) \equiv \text{ind}(x) + \text{ind } y \pmod{\varphi(p)}$

Proof $g^{\text{ind}(xy)} \equiv xy \equiv g^{\text{ind } x} \cdot g^{\text{ind } y} = g^{\text{ind } x + \text{ind } y} \pmod{p}$ 又由于 $a^\gamma \equiv a^{\gamma'} \pmod{m} \Leftrightarrow \gamma \equiv \gamma' \pmod{\delta}$, 得证.

2. $\text{ind } x^c \equiv c \text{ ind } x \pmod{\varphi(p)}$

Proof 由第一个性质直接得到.

3. 若 g_1 也是 p 的原根, 则 $\text{ind}_g a \equiv \text{ind}_{g_1} a \cdot \text{ind}_g g_1 \pmod{\varphi(p)}$.

Proof 设 $x = \text{ind}_{g_1} a \Leftrightarrow g_1^x \equiv a \pmod{p}$, $y = \text{ind}_g g_1 \Leftrightarrow g^y \equiv g_1 \pmod{p}$. 于是我们得到, $a = g^{xy} \equiv g^{\text{ind}_g a}$ (mod p), 利用阶的性质得证.

可以利用 BSGS (Baby Step Giant Step) 算法求离散对数.

对于 $a, b, m \in \mathbb{Z}_+$, BSGS 可以在 $O(\sqrt{m})$ 的时间内求 $a^x \equiv b \pmod{m}$, 其中 $a \perp m$, 且 $0 \leq x < m$ (注意 m 不一定是素数).

令 $x = sB + t$, $B = \lceil \sqrt{m} \rceil$, $s < B$, $t < B$, 于是 $a^x \equiv a^{sB+t} \equiv b \Leftrightarrow a^{sB} \equiv ba^{-t} \pmod{m}$, 于是我们考虑枚举 $b(a^{-1})^t$ 的值, 存入哈希表中, 接下来枚举左边 $(a^B)^s$ 的值, 如果存在与哈希表中, 那么 $x = sB + t$ 就是一个解. 使用 map 的话, 时间复杂度多一个 $\log$, 也就是 $O(\sqrt{m} \log m)$

```

// min x s.t. a^x=b (mod p) p in PP
ll BSGS(ll a, ll b, ll p) {
    if (b == 1) return 0;
    if (a % p == b % p) return 1;
    a %= p, b %= p;

    ll B = ceil(sqrt(p));
    ll ia = inv(a, p), aB = qmi(a, B, p);
    map<ll, ll> mp;
    for (ll t = 0, val = b; t < B; t++, val = val * ia % p)
        if (!mp[val]) mp[val] = t + 1; // 区分没有初始化的哈希表的值
    for (ll s = 0, val = 1; s <= B; s++, val = val * aB % p)
        if (mp[val]) return s * B + mp[val] - 1;
    return -1; // no sol
}

```

3.3.7. NTT

假设质数 $p \in \mathbb{P}$ 可以表示成 $p = r \cdot 2^l + 1$, g 是 p 的原根, 那么我们可以使用 $g_n = g^{\frac{p-1}{n}}$ 来代替 ω_n , 在此基础上进行的 FFT 就是 NTT. (注意这里的 n 依然是 $n = 2^k, k \leq l$)

这是因为它具有和 $\mathbb{C}$ 上单位根一样良好的性质:

- $g_{2n}^{2k} \equiv g_n^k \pmod{p} (2n < 2^l)$
- $g_{2n}^n \equiv -1 \pmod{p} (2n < 2^l)$
- $\sum_{k=0}^{n-1} g_n^{ik} g_n^{-kj} \equiv \begin{cases} n & \text{if } i=j \\ 0 & \text{otherwise} \end{cases} \pmod{p}$, 其中 $0 \leq i, j < n$

因此, 对于 DFT 和 IDFT g_n 在 $\mathbb{Z}_p$ 下与 ω_n 在 $\mathbb{C}$ 下的推导过程是一致的.

常见模数:

- $65537 = 2^{16} + 1, g = 3, g^{-1} = 21846$
- $998244353 = 119 \cdot 2^{23} + 1, g = 3, g^{-1} = 332748118$
- $1004535809 = 479 \cdot 2^{21} + 1 > 10^9, g = 3, g^{-1} = 334845270$
- $4179340454199820289 = 29 \cdot 2^{57} + 1 > 4 \times 10^{18}, g = 3, g^{-1} = 1393113484733273430$

```
constexpr int N = 4e6 + 5;
constexpr ll P = 998244353;
constexpr ll G = 3;           // primitive root
constexpr ll iG = 332748118; // inv(G)

int n, m;
ll f[N], g[N];
ll L, limit, rev[N];

void NTT(ll a[], int lim, int sign) {
    for (int i = 0; i < lim; i++)
        if (i < rev[i]) swap(a[i], a[rev[i]]);
    ll gn, g, x, y;
    for (ll mid = 1; mid < lim; mid <= 1) {
        gn = qmi((sign == 1 ? G : iG), (P - 1) / (mid < 1));
        for (int r = mid < 1, j = 0; j < lim; j += r) {
            g = 1;
            for (int k = 0; k < mid; k++, g = g * gn % P) {
                x = a[j + k] % P, y = g * a[j + mid + k] % P;
                a[j + k] = (x + y + P) % P, a[j + mid + k] = (x - y + P) % P;
            }
        }
    }
    if (sign == -1) {
        ll inv = qmi(lim, P - 2);
        for (int i = 0; i < lim; i++) a[i] = a[i] * inv % P;
    }
}

void convolve(ll f[], ll g[], int df, int dg) {
    limit = 1, L = 0;
    while (limit <= df + dg) limit <= 1, L++;
    for (int i = df + 1; i < limit; i++) f[i] = 0;
    for (int i = dg + 1; i < limit; i++) g[i] = 0;
    for (int i = 0; i < limit; i++) rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (L - 1));
    NTT(f, limit, 1), NTT(g, limit, 1);
    for (int i = 0; i < limit; i++) f[i] = f[i] * g[i] % P;
    NTT(f, limit, -1);
}

void solve() {
    convolve(f, g, n, m);
    for (int i = 0; i <= n + m; i++) cout << f[i] << NN;
}
```

3.3.8. MTT

NTT 可以大值域但是不可任意模数

FFT 可以任意模数（最后取模即可），但是不可以大值域。

对于 MTT，可以使用三模数 NTT+CRT 或者拆系数 FFT。在这里只介绍 三模数 NTT+CRT

既然我们不能再模 M 的意义下直接 NTT，那我们换个思路：

1. 找几个互素的，满足要求的模数 $p_1, p_2, p_3, l \dots$
2. 分别算出多项式乘积在模这些 NTT 模数下的结果，也就是 $C(x)(\bmod p_1), C(x)(\bmod p_2), C(x)(\bmod p_3), l \dots$
3. 利用 CRT，将这些结果合并，得到乘积 $C(x)$ 在模 $p_1 p_2 p_3 \dots$ 下的结果
4. 只要我们选的模数乘积足够大，就能覆盖 $C(x)$ 的真实的系数范围，那么我们得到的就是 $C(x)$ 的真实系数，最后再将这些系数对于 M 取模即可

```

namespace MTT {
constexpr int N = 4e6 + 5;
using lll = __int128;
using poly = vector<ll>;
constexpr ll P1 = 998244353, G1 = 3;
constexpr ll P2 = 1004535809, G2 = 3;
constexpr ll P3 = 469762049, G3 = 3;

ll qmi(ll a, ll b, ll p) {
    ll res = 1;
    for (ll t = a; b; t = (lll)t * t % p, b >>= 1) if (b & 1) res = (lll)res * t % p;
    return res;
}
poly get(int deg) { return poly(deg + 1, 0); }

template <ll P, ll G>
void NTT(poly& a, int lim, int sign) {
    static int rev[N];
    for (int i = 0; i < lim; i++) {
        rev[i] = (rev[i >> 1] >> 1) | ((i & 1) ? (lim >> 1) : 0);
        if (i < rev[i]) swap(a[i], a[rev[i]]);
    }

    ll iG = qmi(G, P - 2, P);
    for (ll mid = 1; mid < lim; mid <= 1) {
        ll gn = qmi((sign == 1 ? G : iG), (P - 1) / (mid < 1), P);
        for (int j = 0; j < lim; j += (mid < 1)) {
            ll g = 1;
            for (int k = 0; k < mid; k++, g = (lll)g * gn % P) {
                ll x = a[j + k];
                ll y = (lll)g * a[j + mid + k] % P;
                a[j + k] = (x + y) % P;
                a[j + mid + k] = (x - y + P) % P;
            }
        }
    }

    if (sign == -1) {
        ll inv_lim = qmi(lim, P - 2, P);
        for (int i = 0; i < lim; i++) a[i] = (lll)a[i] * inv_lim % P;
    }
}

poly add(poly f, poly g, ll mod){
    int d = max(f.size(), g.size());
    f.resize(d), g.resize(d);
    poly res = get(d - 1);
    for (int i = 0; i < d; ++i) res[i] = (f[i] + g[i]) % mod;
    return res;
}

poly sub(poly f, poly g, ll mod) {
    int d = max(f.size(), g.size());
    f.resize(d), g.resize(d);
    poly res = get(d - 1);
    for (int i = 0; i < d; ++i) res[i] = (f[i] - g[i] + mod) % mod;
    return res;
}

poly mul(poly f, poly g, int mod) {
    if (f.empty() || g.empty()) return {};

    int df = f.size() - 1, dg = g.size() - 1;
    int limit = 1;
    while (limit <= df + dg) limit <= 1;

    f.resize(limit), g.resize(limit);

```

```

poly f1 = f, g1 = g;
for(auto& x : f1) x %= P1; for(auto& x : g1) x %= P1;
NTT<P1, G1>(f1, limit, 1); NTT<P1, G1>(g1, limit, 1);
for (int i = 0; i < limit; i++) f1[i] = (lll)f1[i] * g1[i] % P1;
NTT<P1, G1>(f1, limit, -1);

poly f2 = f, g2 = g;
for(auto& x : f2) x %= P2; for(auto& x : g2) x %= P2;
NTT<P2, G2>(f2, limit, 1); NTT<P2, G2>(g2, limit, 1);
for (int i = 0; i < limit; i++) f2[i] = (lll)f2[i] * g2[i] % P2;
NTT<P2, G2>(f2, limit, -1);

poly f3 = f, g3 = g;
for(auto& x : f3) x %= P3; for(auto& x : g3) x %= P3;
NTT<P3, G3>(f3, limit, 1); NTT<P3, G3>(g3, limit, 1);
for (int i = 0; i < limit; i++) f3[i] = (lll)f3[i] * g3[i] % P3;
NTT<P3, G3>(f3, limit, -1);

poly res(df + dg + 1);
ll invp12 = qmi(P1, P2 - 2, P2);
ll invp123 = qmi((lll)P1 * P2 % P3, P3 - 2, P3);
ll M1 = P1;
ll M2 = (lll)P1 * P2;

for (int i = 0; i <= df + dg; i++) {
    ll a1 = f1[i], a2 = f2[i], a3 = f3[i];
    ll k1 = (lll)(a2 - a1 + P2) % P2 * invp12 % P2;
    ll A = a1 + k1 * P1;
    ll k2 = (lll)(a3 - A % P3 + P3) % P3 * invp123 % P3;
    res[i] = ((lll)k2 * (M2 % mod) % mod + A % mod) % mod;
}
return res;
}
}
using namespace MTT;

```

3.3.9. Newton's Method

牛顿迭代法可以解决：给定多项式 $g(x)$ ，求满足 $g(f(x)) = 0$ 的形式幂级数 $f(x) = \sum_{i=0}^{\infty} a_i x^i$

1. 当 $n = 1$ ，也就是求 $[x^0]f(x)$ 时，解 $g(a_0) = 0$
2. 假设已经求出了前 n 项 $f_0(x) \equiv a_0 + a_1x + \dots + a_{n-1}x^{n-1} \pmod{x^n}$ ，则 $f(x) \equiv f_0(x) - \frac{g(f_0(x))}{g'(f_0(x))} \pmod{x^{2n}}$

Proof:

1. 设 $g(x) = \sum_{i=0}^n b_i x^i$ ，则 $[x^0]g(f(x)) = b_0 + b_1 a_0 + \dots = 0 = g(a_0)$
2. 考虑对 $g(x)$ 进行泰勒展开

$$\begin{aligned}
 g(f(x)) &= g(f_0(x)) + g'(f_0(x))(f(x) - f_0(x)) \\
 &\quad + \frac{g''(f_0(x))}{2!}(f(x) - f_0(x))^2 + \dots + \frac{g^{(n)}(f_0(x))}{n!}(f(x) - f_0(x))^n + \dots
 \end{aligned} \tag{1}$$

由于 $f(x) - f_0(x) = x^n(a_n + a_{n+1}x + \dots)$ ，因此 $(f(x) - f_0(x))^2 \equiv 0 \pmod{x^{2n}}$ 因此在 (1) 中，可以变成

$$\begin{aligned}
 g(f(x)) &\equiv 0 \equiv g(f_0(x)) + g'(f_0(x))(f(x) - f_0(x)) \pmod{x^{2n}} \\
 \Leftrightarrow f(x) &\equiv f_0(x) - \frac{g(f_0(x))}{g'(f_0(x))} \pmod{x^{2n}}
 \end{aligned}$$

3.3.9.1. poly inverse

设 $h(x)$ 为给定的形式幂级数，求他的逆 $f(x)$ 则定义 $g(f(x)) = \frac{1}{f(x)} - h(x) = 0$ ，得到的迭代式为

$$f(x) \equiv 2f_0(x) - f_0^2(x)h(x) \equiv f_0(x)(2 - f_0(x)h(x)) \pmod{x^{2n}}$$

时间复杂度 $O(n \log n)$

3.3.9.2. poly square root

设 $h(x)$ 为给定的形式幂级数, 求他的平方根 $f(x)$ 则 定义 $g(f(x)) = f(x)^2 - h(x) = 0$, 得到的迭代式为

$$f(x) \equiv f_0(x) - \frac{f_0^2(x) - h(x)}{2f_0(x)} \equiv \frac{f_0(x)^2 + h(x)}{2f_0(x)} \equiv 2^{-1} \cdot (f_0(x) + h(x)f_0^{-1}(x)) \pmod{x^{2n}}$$

时间复杂度 $O(n \log n)$

3.3.9.3. poly deriavative

假设 $f(x) = \sum_{i \geq 0} a_i x^i$, 定义 $f'(x) = \frac{df}{dx} = \sum_{i \geq 0} a_{i+1} x^i$

时间复杂度 $O(n)$

3.3.9.4. poly intergral

假设 $f(x) = \sum_{i \geq 1} a_i x^i$, 定义 $\int f dx = \sum_{i \geq 0} \frac{a_{i+1}}{i+1} x^{i+1}$, 在这里我们不考虑 $+C$

时间复杂度 $O(n)$

3.3.9.5. poly composition

假设 $f(x) = \sum_{i \geq 0} a_i x^i, g(x) = \sum_{i \geq 0} b_i x^i$ 则 $g(f(x)) = f \circ g(x) = \sum_{i \geq 0} b_i f(x)^i = \sum_{i \geq 0} c_i x^i$, 其中 $c_0 = b_0, c_n = \sum_{k=1}^n b_k \sum_{i_1+i_2+\dots+i_k=n} a_{i_1} a_{i_2} \dots a_{i_k}$

注意在此处的讨论, 我们会将形式幂级数的常数项默认设为 0, 为了让他的性质更好

3.3.9.6. poly logarithm

$$\bullet \ln(1+x) = \sum_{n \geq 1} \frac{(-1)^{n+1}}{n} x^n$$

若果形式幂级数 $f(x)$ 满足常数项为 0, 则可以定义 $\ln(f(x)) := \ln(1+f(x))$

于是, 我们有

$$(\ln(f(x)))' = f'(x) \frac{1}{f(x)}$$

时间复杂度 $O(n \log n)$

3.3.9.7. poly exp

$$\bullet \exp(x) = \sum_{n \geq 0} \frac{1}{n!} x^n$$

若果形式幂级数 $f(x)$ 满足常数项为 0, 则可以定义 $\exp(f(x))$

我们假设 $g(x) = \exp(f(x))$, 则 $\ln(g(x)) - f(x) = 0$, 构造 $h(x, f) = \ln x - f(x) = \frac{1}{x} - f(x)$, 利用牛顿迭代, 假设得到了 $\text{mod } x^n$ 意义下的 $g_0(x)$, 那么

$$g(x) \equiv g_0(x)(1 - \ln(g_0(x)) + f(x)) \pmod{x^{2n}}$$

时间复杂度 $O(n \log n)$

更加完善的 NTT 多项式代码

```

////////// NTT poly //////////
constexpr ll MOD = 998244353;
namespace Poly {
constexpr int N = 4e6 + 5;
constexpr ll P = 998244353;
constexpr ll G = 3; // primitive root
constexpr ll iG = 332748118; // inv(G)

ll qmi(int a, int b) {
    ll res = 1;
    for (ll t = a; b; t = t * t % P, b >>= 1)
        if (b & 1) res = res * t % P;
    return res;
}

void NTT(vi &a, int lim, int sign, const ll rev[]) { // lim=2^k

```

```

    for (int i = 0; i < lim; i++)
        if (i < rev[i]) swap(a[i], a[rev[i]]);
    ll gn, g, x, y;
    for (ll mid = 1; mid < lim; mid <= 1) {
        gn = qmi((sign == 1 ? G : iG), (P - 1) / (mid < 1));
        for (int r = mid < 1, j = 0; j < lim; j += r) {
            g = 1;
            for (int k = 0; k < mid; k++, g = g * gn % P) {
                x = a[j + k] % P, y = g * a[j + mid + k] % P;
                a[j + k] = (x + y + P) % P, a[j + mid + k] = (x - y + P) % P;
            }
        }
    }
    if (sign == -1) {
        ll inv = qmi(lim, P - 2);
        for (int i = 0; i < lim; i++) a[i] = a[i] * inv % P;
    }
}
using poly = vi;
poly get(int deg) { return poly(deg + 1, 0); }
poly operator+(poly f, poly g) {
    int d = max(f.size(), g.size());
    f.resize(d), g.resize(d);
    poly res = get(d - 1);
    for (int i = 0; i < d; ++i) res[i] = (f[i] + g[i]) % P;
    return res;
}
poly operator-(poly f, poly g) {
    int d = max(f.size(), g.size());
    f.resize(d), g.resize(d);
    poly res = get(d - 1);
    for (int i = 0; i < d; ++i) res[i] = (f[i] - g[i] + P) % P;
    return res;
}
poly operator*(poly f, poly g) {
    if (f.empty() || g.empty()) return {};
    static ll rev[N];
    int df = f.size() - 1, dg = g.size() - 1, limit = 1, L = 0;
    while (limit <= df + dg) limit <= 1, L++;
    for (int i = 0; i < limit; i++) rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (L - 1));
    f.resize(limit), g.resize(limit);
    NTT(f, limit, 1, rev), NTT(g, limit, 1, rev);
    for (int i = 0; i < limit; ++i) f[i] = f[i] * g[i] % P;
    NTT(f, limit, -1, rev);
    poly res = get(df + dg);
    ll inv = qmi(limit, P - 2);
    for (int i = 0; i <= df + dg; i++) res[i] = f[i];
    return res;
}
poly operator*(poly f, ll x) {
    for (int i = 0; i < f.size(); i++) f[i] = f[i] * x % P;
    return f;
}
poly mod(poly f, int n) {
    f.resize(n, 0);
    return f;
}
poly inv(const poly& h) {
    poly f = get(0), d = get(0);
    f[0] = qmi(h[0], P - 2), d[0] = 2;
    ll n = h.size();
    for (int len = 2; (len >> 1) < n; len <= 1) // f0(x) 已经有了 len>>1 项, 接下来是要 mod x^(len)
        f = mod(f * (d - mod(h, len) * f), len);
    return f;
}
poly sqrt(const poly& h) {
    poly f = get(0);
    f[0] = 1;

```



```

ll inv2 = qmi(2, P - 2), n = h.size();
for (int len = 2; (len >> 1) < n; len <= 1) {
    poly exf = f;
    exf.resize(len);
    poly invf = inv(exf);
    poly t = mod(h, len);
    t = mod(t * invf, len);
    f.resize(len);
    f = (f + t) * inv2;
}
return f;
}
poly dervt(const poly& f) {
    if (f.empty()) return poly();
    int n = f.size() - 1;
    if (n == 0) return get(-1); // Derivative of a constant is 0
    poly res = get(n - 1);
    for (int i = 0; i < n; i++)
        res[i] = (ll)f[i + 1] * (i + 1) % P;
    return res;
}
poly integ(const poly& f) {
    int n = f.size() - 1;
    poly res = get(n + 1);
    for (int i = 0; i <= n; i++)
        res[i + 1] = (ll)f[i] * qmi(i + 1, P - 2) % P;
    return res;
}
poly ln(const poly& f) {
    int n = f.size();
    assert(n == 0 || f[0] == 1);
    poly t = dervt(f) * inv(f);
    return integ(mod(t, n - 1));
}
poly exp(const poly& f) {
    int n = f.size();
    assert(n == 0 || f[0] == 0);
    poly B = get(0);
    B[0] = 1;
    for (int len = 2; (len >> 1) < n; len <= 1) {
        poly lnB = ln(mod(B, len)), t = mod(f, len) - lnB;
        t[0] = (t[0] + 1 + P) % P;
        B = B * t;
        B.resize(len);
    }
    B.resize(n);
    return B;
}
} // namespace Poly
using namespace Poly;

```

3.4. Combinatorics

3.4.1. binomial inversion

二项式反演 常用于通过“指定若干个”求“恰好若干个”的问题。

考虑容斥原理基本形式

$$|A_1^c \cap A_2^c \cap \dots \cap A_n^c| = |S| - \sum_{1 \leq j_1 \leq n} |A_{j_1}| + \sum_{1 \leq j_1 < j_2 \leq n} |A_{j_1} \cap A_{j_2}| + \dots + (-1)^n \left| \bigcap_{i=1}^n A_{i_1} \right|$$

两边替换为补集，得到

$$|A_1 \cap A_2 \cap \dots \cap A_n| = |S| - \sum_{1 \leq j_1 \leq n} |A_{j_1}^c| + \sum_{1 \leq j_1 < j_2 \leq n} |A_{j_1}^c \cap A_{j_2}^c| + \dots + (-1)^n \left| \bigcap_{i=1}^n A_{i_1}^c \right|$$

如果，只和集合的数量有关，而不是具体的集合，那么将 n 个补集集合的交的大小记作 $f(n)$ ，将 n 个原集的集合的交的大小记作 $g(n)$ ，可以得到

$$f(n) = \sum_{i=0}^n (-1)^i \binom{n}{i} g(i) \Leftrightarrow g(n) = \sum_{i=0}^n (-1)^i \binom{n}{i} f(i)$$

与莫比乌斯反演类似，二项式反演有两种：

- 对子集反演 $f(n) = \sum_{i=m}^n \binom{n}{i} g(i) \Leftrightarrow \sum_{i=m}^n (-1)^{n-i} \binom{n}{i} f(i)$
- 对超集反演 $f(n) = \sum_{i=n}^m \binom{i}{n} g(i) \Leftrightarrow g(n) = \sum_{i=n}^m (-1)^{m-i} \binom{i}{n} f(i)$

3.4.2. Stirling number of the 2nd kind

第二类斯特林数 $S_2(n, k) = \left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$ 表示 n 个不同的小球放入 k 个相同的盒子（每个盒子不能为空）的方案数。

1. 递推关系： $S_2(n, k) = S_2(n-1, k-1) + k \cdot S_2(n, k-1)$
2. 通项公式： $S_2(n, k) = \frac{1}{k!} \sum_{i=0}^k (-1)^i \binom{k}{i} (k-i)^n$
3. 重要公式： $x^n = \sum_{k=0}^n S_2(n, k) x^k$
4. 关于 n 的 EGF： $\sum_{n \geq 0} S_2(n, k) \frac{x^n}{n!} = \frac{1}{k!} (\exp(x) - 1)^k$

Proof:

1. 考虑第 k 个球：如果新放入一个盒子，那么前面的球只能放入 $n-1$ 个盒子；如果第 k 个球放入已有的盒子，那么一定有了 k 个盒子，而当前的球可以随便放置其中。
2. 先假设这 k 个盒子互相区分，假设 a_i 表示编号为 i 的盒子为空，那么将 n 个球放到其中的方案数利用容斥可以得到是 $N((1-a_1)(1-a_2)\cdots(1-a_k)) = \sum_{i=0}^k (-1)^i \binom{k}{i} N(a_1 a_2 \cdots a_i) = \sum_{i=0}^k (-1)^i \binom{k}{i} (k-i)^n$ ，最后再把盒子的编号消除，得证。
3. 考虑 n 个球放到 x 个互相区分的盒子中（盒子可以为空），那么 LHS 是显然的，来看 RHS，考虑枚举非空的盒子的数量假设为 k 个，那么选出这 k 个盒子的方案数为 x^k ，对于每一种选出来的方案，放小球的方案数为 $S_2(n, k)$ ，得证。
4. LHS 就是 S_2 的 EGF，我们看 RHS，假设盒子是互相区分的，将 n 个球放入一个盒子中的 EGF 就是 $\exp(x) - 1$ ，因为 0 个球方案是 0，而其他情况都只有一种方案。由于盒子是互相区分的，因此放入 k 个盒子的 EGF 就是 $(\exp(x) - 1)^k$ ，再消除盒子的顺序，得证。

快速求 $S_2(n, k)$

- 一行 $k = 0, 1, \dots, n$
考虑通项公式

$$S_2(n, k) = \frac{1}{k!} \sum_{i=0}^k (-1)^i \binom{k}{i} (k-i)^n = \sum_{i=0}^k \frac{(-1)^i}{i!} \cdot \frac{(k-i)^n}{(k-i)!}$$

于是，变成了卷积相乘后的 k 次项，时间复杂度 $O(n \log n)$

```
constexpr int N = 2e5 + 5;
ll ivf[N];
void solve() {
    ll n, k; cin >> n >> k;

    ivf[0] = ivf[1] = 1;
    for (int i = 2; i <= n; i++) ivf[i] = ivf[i-1] * qmi(i, MOD-2) % MOD;

    poly f = get(n), g = get(n);
    for (int i = 0; i <= n; i++) {
        if (i & 1) f[i] = (-ivf[i] + MOD) % MOD;
        else f[i] = ivf[i] % MOD;
        g[i] = qmi(i, n) * ivf[i] % MOD;
    }

    auto ans = f * g;
    for (int i = 0; i <= n; i++) cout << ans[i] << " ";
}
```

- 一列 $n = 0, 1, \dots, N$

考虑关于 n 的 EGF: $\sum_{n \geq 0} S_2(n, k) \frac{x^n}{n!} = \frac{1}{k!} (\exp(x) - 1)^k$, 因此对于给定的 k 可以直接求出 $\frac{S_2(n, k)}{n!}$, 时间复杂度 $O(n \log n)$

注意对多项式快速幂中取 $\ln$ 操作, 要求 $[x^0]f(x) = 1$, 因此在处理 $\exp(x) - 1$ 时, 可以考虑转化成 $\exp(x) - 1 = x + \frac{1}{2!}x^2 + \dots + \frac{1}{n!}x^n + \dots = x(1 + \frac{1}{2!}x + \dots + \frac{1}{n!}x^{n-1})$, 然后再分成两部分乘法即可。

```
constexpr int N = 131080;
ll ivk, fac[N];
void solve(){
    ll n, k; cin >> n >> k;

    fac[0] = 1;
    for (int i = 1; i <= n; i++) fac[i] = fac[i - 1] * i % MOD;

    poly g = get(n);
    for (int i = 0; i <= n; i++) g[i] = qmi(fac[i + 1], MOD - 2);
    auto lnG = ln(g);
    lnG = lnG * k;
    g = exp(lnG);

    ll ivk = qmi(fac[k], MOD - 2);
    for (int i = 0; i <= n; i++) {
        if (i < k) cout << 0 << NN;
        else cout << fac[i] * ivk % MOD * g[i - k] % MOD << NN;
    }
}
```

3.4.3. partition number

k 部分拆数 $p(n, k)$ 表示 n 个相同的小球放入 k 个相同的盒子的方案数。

1. 递推关系: $p(n, k) = p(n - 1, k - 1) + p(n - k, k)$
2. 关于 n 的 OGF: $\sum_{n \geq 0} p(n, k) x^n = x^k \prod_{i=1}^k \frac{1}{1 - x^i}$

3.4.4. Stirling number of the 1st kind

4. References

- alex-wei - 基础图论
- 莫比乌斯反演入门 - 知乎
- 「笔记」高中生都能看懂的莫比乌斯反演
- 初探莫比乌斯反演及欧拉反演
- 莫比乌斯反演-让我们从基础开始 - 洛谷
- 莫比乌斯反演-从莫比乌斯到欧拉 - 洛谷
- Peter 的莫比乌斯反演与各种筛法题单 - 洛谷
- 模运算和同余以及欧几里得
- P1495 CRT,P4777 EXCRT - suxxsfe
- 糖老师 blog - 浅谈一类积性函数的前缀和
- 论逗逼的自我修养之寒假颓废记
- 杜教筛
- 數論 狄利克雷卷积 & 莫比烏斯反演
- 数论分块 - OI Wiki
- 线性基 - OI Wiki
- 关于下降幂
- 快速傅里叶变换(FFT)详解
- FFT/NTT 总结
- FFT(快速傅里叶变换)0 基础详解! 附 NTT (ACM/OI) - 知乎
- Algorithm: 多项式乘法 Polynomial Multiplication: 快速傅里叶变换 FFT / 快速数论变换 NTT
- 拉格朗日插值法及应用
- P6091 【模板】原根 题解
- FFT / NTT / MTT 学习笔记
- 多项式与生成函数学习笔记
- EntropyIncreaser's Blog
- 炫酷反演魔术
- FFT/NTT 基本应用
- 二项式反演及其应用
- 初中生都看得懂的快速上手斯特林数指南——从盒放球问题说起
- 多项式计数杂谈